Understanding Backpropagation in Convolutional Neural Networks

A Tutorial Using Heart and Lung Sound Classification

Name: Francis Chisom Ofonagoro
Student ID: 24101661
Module: Machine Learning and Neural Networks

Introduction

Backpropagation is a type of supervised learning training algorithm that is used in neural networks to train them based on some given input data and on learning from their mistakes and correcting them based on the error in the output. Backpropagation is used to compute the parameters in the model by updating the parameters based on the prediction error. The Backpropagation training process involves backpropagating errors from the output layer to the input layer, then computing the gradients of the loss function with respect to the network's weights. This is done by using the method of taking the derivative of the function, which is known as the Chain Rule.

This note focuses on how we use gradients to "learn" or update the values of the weights and neurons to yield better outcomes for the model. Using gradients in backpropagation informs us formally of how the loss function changes given a change in the value of some parameter. This sensitivity helps us to understand, approximately, the magnitude of contribution of each of the weights to the output (or the error in that output, given that's what we're often trying to minimize). The gradient can then be used within the update rule for some optimization algorithm like Stochastic Gradient Descent.

At an intuitive level, backpropagation answers two fundamental questions:

- **How wrong was the model's prediction?**
- **How should each parameter be adjusted to reduce this error?**

Every training iteration involves the forward pass, which involves inputting the data into the network, with the output of the forward pass resulting in a predicted output value. The backward pass then involves iterating back through the network to apply the gradients of the losses to optimise the weights and topology of the network. The learning phase involves applying these updates to the network for use in the subsequent iteration.

Ultimately, after many iterations of the process, the network is much better able to extract features and make more accurate output predictions.

Why Backpropagation is Needed in any Neural Network

Backpropagation is needed for four major reasons:

- Backpropagation identifies the source of prediction error by tracing the error backwards through the output to inputs and identifying which weights have the most influence on the error.

- Autograd allows us to compute gradients of our loss function with respect to parameters in our model, so we can then use an optimizer to update the weights and biases based on those gradients. These gradients describe the change that each weight and bias needs to undergo in order to reduce our loss.

- It trains a neural network to learn across multiple layers by taking the error from the output layer and sending it back to the input layer and all the hidden layers, so that the network is improved as a whole.

- This allows the training of large neural networks by the calculation of gradients for 1000's or even millions of parameters in a single backward pass, where this calculation would be cumbersome if done for each parameter separately.

Architecture Context (Where Backpropagation Fits)

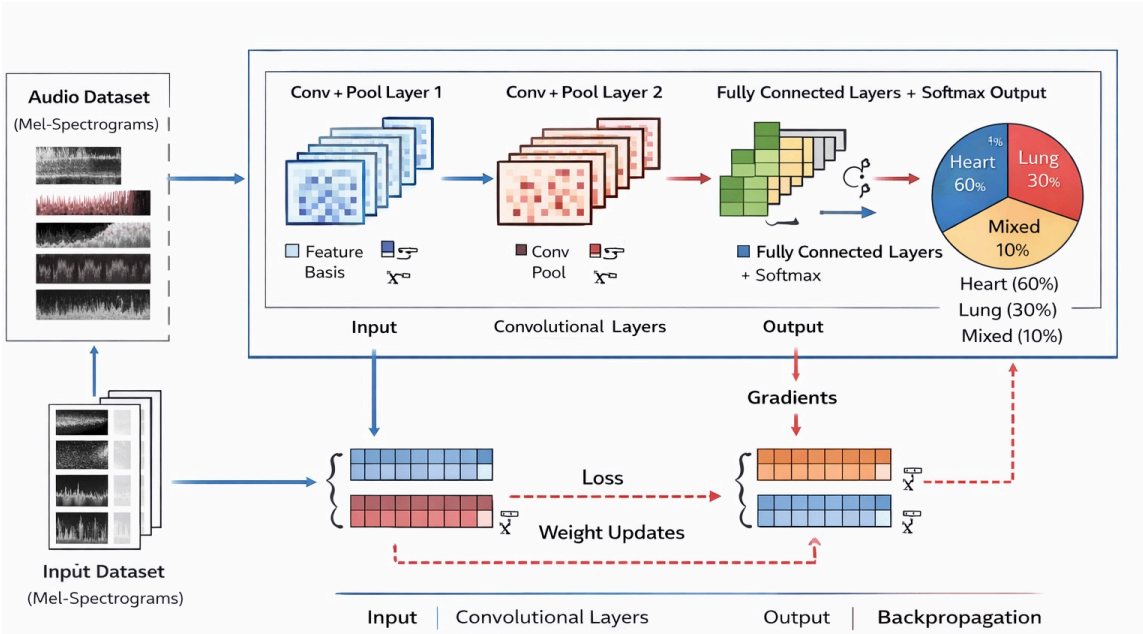

Figure 1: Convolutional Neural Network Architecture and Backpropagation Flow for Mel-Spectrogram Classification

A Convolutional Neural Network (CNN) or ConvNet performs a computation task where learning takes place in two phases – the Forward pass and the Backward pass.

During the **forward pass**, the input data is fed into each layer in succession. This involves passing through the convolutional layers, pooling layers, and finally into the fully connected dense layers to produce an output prediction. In this project, the input to the model is a Mel-spectrogram of audio data. This is passed through the convolutional layers to obtain the local time–frequency features, the pooling layers for spatial reduction, and the dense layers for the combination of the extracted features to make a class prediction. In summary, the forward pass is responsible for the inference of the output from the input features.

The **backward pass**, also known as the backpropagation process, starts after the network predictions are compared to the actual class label by the loss function. The network then propagates the error backward to compute the gradients of the loss with respect to the network parameters. The gradients give us the information about the amount of

contribution of each weight in the network to the total error. The network parameters are then updated based on the gradients. This whole process helps the network to learn how to minimize future errors. It does so by adjusting the weights and biases in the network

Backpropagation in CNN Layers

Some of the layers we can commonly find in most neural networks:

1. **Convolution Layer**

A convolutional layer learns spatially localised features from input data using a set of trainable filters. During the backpropagation phase, gradients of the loss with respect to filter weights are calculated using the convolution operation of input activations and the gradients of the previous layer.

A key feature of a convolutional layer is weight sharing, that is, a filter is used to slide over the input, feature map, or output space. In the backpropagation phase, gradients from all the spatial locations in the output feature map are summed up and used to update the shared filter. This sharing of weights between different features leads to the reduction in the number of parameters and also helps in the regularization of the model.

2. **Activation Function (ReLU)**

Also known as the activation function of the neural network. The Rectified Linear Unit (ReLU) is a linear function with a non-linear operation. The output obtained after the ReLU function is applied has more smoothness, and it helps in bringing non-linearity in the network. It also affects the gradient flow in the network. The derivative of the ReLU function is:

$$f'(x) = \{\begin{matrix} 1 & x > 0 \\ 0 & x \leq 0 \end{matrix}$$

The ReLU derivative acts as a gate to filter the gradients during the backpropagation phase. It lets the gradients through for the positive activation values, while blocking the gradients for the non-positive activation values, thus reducing the vanishing gradient problem and helping in a faster training of deep neural networks.

3. **Pooling Layer**

The Pooling Layer is a layer in a deep neural network where spatial down-sampling takes place, and they do not contain learnable parameters. Therefore, they do not require the computation of gradients with respect to weights. Instead, the gradients are simply passed on to previous layers in the backpropagation phase.

In max pooling, the gradient of the loss with respect to. The input is only backpropagated to the input element responsible for the maximum value during the forward pass. All other elements in the input receive a gradient of 0, which ensures that the most important features detected during the feature extraction stage are preserved.

4. **Fully Connected (Dense) Layer**

The fully connected layer behaves just like a standard neural network layer where each neuron is fully connected to all activations in the previous layer. During the backpropagation phase, the gradients are calculated using the chain rule just like with feedforward networks.

The convolution operation applied in each layer is defined as:

$$X^{(l)} = f(W^{(l)} * X^{(l-1)} + b^{(l)})$$

Where

$X^{(l-1)}$is the input,
$X^{(l)}$ represents the output,

$W^{(l)}$represents convolutional filters,

f represents the activation function and b(l) represents the bias.

Backpropagation Algorithm (Step-by-Step)

Step 1: **Forward Pass**

- Input spectrogram → CNN → prediction

Step 2: **Compute Loss**

- Compare predicted vs actual label

Step 3: **Compute Gradients**

- Calculate how much each weight contributed to error

Step 4: **Backward Pass**

- Propagate gradients from output → input

Step 5: **Update Weights**

- Apply gradient descent

Step 6: **Repeat**

- Continue for multiple epochs

Mathematical Foundation

Loss Function

For classification, we use **Cross-Entropy Loss**:

$$L = -\sum_{i} \quad y_i \log(\hat{y}_i)$$

It measures how wrong the prediction is.

Weight Update Rule

$$w = w - \eta \frac{\partial L}{\partial w}$$

Where:

- η= learning rate
- $\frac{\partial L}{\partial w}$= gradient

Chain Rule (Derivative of the gradient)

$$\frac{\partial L}{\partial W^{(l)}} = \frac{\partial L}{\partial A^{(l)}} \cdot \frac{\partial A^{(l)}}{\partial Z^{(l)}} \cdot \frac{\partial Z^{(l)}}{\partial W^{(l)}}$$

This allows error to flow layer by layer.

Output layer (Softmax)

The model produces output probabilities using the softmax function:

$$P(y_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

Experiment: Backpropagation Application

In today's popular deep learning frameworks, such as TensorFlow, backpropagation is taken care of by higher-level APIs during the training process of a deep learning model. In this project, we have to describe the training process using the gradients that flow from the output backwards to the input of a layer. From the model I developed:

```
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)
```

Role of the Optimizer (Adam)

Optimizer with adaptive learning rate and momentum. The Adam optimizer is a first-order optimizer that is inspired by advanced versions of stochastic gradient descent methods. This algorithm is a variant of stochastic gradient descent (SGD) that can adapt the learning rate for each parameter individually, based on the estimates of the first and second moments of the gradients. This results in faster convergence and stability while training.

These gradients are then used by the optimizer (Adam in this case) to update the model parameters:

$$W_{t+1} = W_t - \eta \frac{\partial L}{\partial W}$$

```
history = model.fit(
    X_train,
    y_train,
    validation_split=0.2,
    epochs=10,
    batch_size=16,
    callbacks=[early_stop],
    verbose=1,
)
```

In the backpropagation, the `model.fit()` function is used to train the model. When we use `model.fit()`, backpropagation is applied in the background to train the model and minimize the loss function.

Internal Mechanism of Backpropagation

Under the hood, TensorFlow utilises **automatic differentiation** to compute gradients efficiently. This process is conceptually equivalent to the following:

```
[8]
✓ 1s
loss_fn = tf.keras.losses.SparseCategoricalCrossentropy()
optimizer = tf.keras.optimizers.Adam()

x_batch = tf.convert_to_tensor(X_train[:16])
y_batch = tf.convert_to_tensor(y_train[:16])

with tf.GradientTape() as tape:
    predictions = model(x_batch, training=True)
    loss_value = loss_fn(y_batch, predictions)

gradients = tape.gradient(loss_value, model.trainable_variables)
```

Training/Validation Process

The training process involves minimising a loss function L, defined for classification tasks (such as this project) using sparse categorical cross-entropy:

$$L = -\sum y \log(\hat{y})$$

Figure 2: Training and Validation Loss Across Epochs.

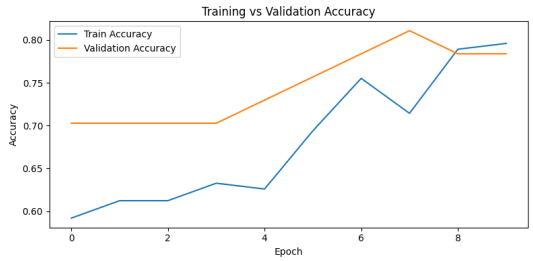

Figure 3: Training and Validation Accuracy Across Epochs

Backpropagation computes the gradient of the loss with respect to each trainable parameter:

$$\frac{\partial L}{\partial W}$$

Limitations of Backpropagation

Overfitting: A problem with the backpropagation algorithm is that the model may learn nothing but noise and characteristics unique to the training set. These learned characteristics do not generalize well to unseen examples of the class, particularly if the training set is small and/or not diverse.

Vanishing Gradients: When you look at the gradients flowing through the layers in a deep neural network, the first thing that stands out is that the gradients for the earlier layers end up being very small. What follows is that the actual weight updates for these early layers also end up being very small, which results in the network not really learning anything from these gradients.

Data Dependency: One critical factor that determines the success of the backpropagation algorithm is the data we are training and the corresponding quality, adequacy, and representativeness of the training data.

Solutions and Improvements

- Regularisation Techniques: implementing any of the techniques for regularisation, such as Dropout, L2 Regularisation, and stopping early when training to avoid overfitting.
- Stabilising Gradient Flow:

ReLU Activation ReLU (Rectified Linear Unit

He/Xavier Initialisation

Batch Normalisation: The Batch Normalisation normalises the input of the layer. The main goal of Batch Normalisation is to stabilise and speed up the training process as well as to improve the gradient flow.

- Data Quality & Enhancement:

Data Augmentation is a technique that increases the amount of training data as well as its diversity. It creates new training data from the existing data. Some of the common techniques used for Data Augmentation include, but are not limited to, adding random noise to the data, a time shift in audio data.

Balanced Dataset

Summary

This all works by utilising an optimisation method called Backpropagation. This is a form of gradient descent optimisation algorithm. In this case it is used in the CNN to actually learn what the features that make up the Mel-spectrograms are. As you can guess backpropagation is the "back" in the Backpropagation, it is called this because it works by reversing the chain rule for efficient computation of the gradients and update of the weights. We also make use of this method within our project to enable our CNN to actually learn these Mel-spectrograms from our audio data, and as you can see the results have not given us the output we required, so after further inspection we found that there was a lot of overfitting occurring within our CNN, meaning we need some form of regularisation method as well as a larger dataset to really test this out, as deep learning requires a lot of data.

References

Goodfellow, I., Bengio, Y. and Courville, A., 2016. *Deep Learning*. MIT Press.

Rumelhart, D.E., Hinton, G.E. and Williams, R.J., 1986. Learning representations by back-propagating errors. *Nature*, 323(6088), pp.533–536.

LeCun, Y., Bottou, L., Bengio, Y. and Haffner, P., 1998. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), pp.2278–2324.

TensorFlow, 2023. *Automatic differentiation and GradientTape*. Available at: https://www.tensorflow.org/guide/autodiff

Kingma, D.P. and Ba, J., 2015. Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR)*.

Nair, V. and Hinton, G.E., 2010. Rectified linear units improve restricted Boltzmann machines. *ICML*.